

Guía Docente Diapositiva por Diapositiva — Sesión 2: Dockerfile Profesional, Imágenes y Docker Hub

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026)

Instructor: Ing. Cristian Jampier Chileno Segundo | OTI - UNI

Programa: Programa de Iniciación Tecnológica (PIT 2026) — Universidad Nacional de Ingeniería

Total Diapositivas: 34 Diapositivas

Instrucciones de Orientación Pedagógica

Esta guía contiene la explicación detallada y el guión profesional en primera persona para abordar **cada una de las 34 diapositivas** de la presentación oficial de la Sesión 2. Está diseñada para guiar la clase paso a paso, garantizando que los estudiantes aprendan a escribir recetas de imágenes profesionales (Dockerfiles), comprendan el sistema de capas y caché, optimicen tamaños con Multi-stage y `.dockerignore`, y dominen la publicación en Docker Hub.

Explicación Diapositiva por Diapositiva (1 a 34)

 Diapositiva 1: DOCKER DESDE CERO: Crea y Despliega Aplicaciones — Sesión 2

Contenido de la PPT:

DOCKER DESDE CERO: Crea y Despliega Aplicaciones
INSTRUCTOR: Cristian Jampier Chileno Segundo
PROGRAMA DE INICIACIÓN TECNOLÓGICA – PIT 2026
Oficina de Tecnologías de la Información (OTI - UNI)
Programa Completo – PIT 2026

Guión del Docente (Lo que debes decir a los alumnos):

"Muy buenos días/tardes a todos. Bienvenidos a la **Sesión 2** del curso *Docker desde Cero: Crea y Despliega Aplicaciones*, impartido por la OTI-UNI. En la sesión anterior aprendimos a ejecutar contenedores desde imágenes existentes en Docker Hub. Hoy daremos el salto técnico fundamental: aprenderemos a escribir nuestras propias recetas profesionales (**Dockerfiles**), entenderemos cómo funciona el sistema de capas inmutables y el caché de compilación, y publicaremos nuestras imágenes en Docker Hub."

Acción en Consola / Pizarra:

- Proyectar la portada del curso y recordar la ruta del repositorio oficial:
<https://github.com/Crsitian22/docker-desde-cero-pit>.

Tip de Gestión del Aula:

- Preguntar al aula: "¿Quiénes pudieron completar la tarea de la Sesión 1 de correr Nginx y Apache en paralelo?"

Diapositiva 2: SESIÓN 2 — Índice del Temario

Contenido de la PPT:

SESIÓN 2

1. Anatomía del Dockerfile
2. Instrucciones Clave
3. Capas y Caché
4. Buenas Prácticas y `.dockerignore`
5. Registro e Imágenes Base
6. Publicación en Docker Hub

Guión del Docente (Lo que debes decir a los alumnos):

"En esta segunda sesión cubriremos 6 bloques estructurados:

1. Conoceremos la **Anatomía del Dockerfile**.
2. Analizaremos a fondo las **Instrucciones Clave**: `FROM`, `RUN`, `COPY`, `WORKDIR`, `ENV`, `EXPOSE`, `CMD` y `ENTRYPOINT`.
3. Entenderemos la magia de las **Capas y el Caché** del motor de Docker.
4. Aplicaremos **Buenas Prácticas** y el uso del archivo `.dockerignore`.
5. Compararemos **Imágenes Base** (`slim` vs `alpine`) y el patrón **Multi-stage build**.
6. Y finalmente, realizaremos la **Publicación en Docker Hub** con tagging y versionado profesional."

Acción en Consola / Pizarra:

- Anotar el mapa conceptual en la esquina de la pizarra: `Dockerfile` -> `Build` -> `Imagen` -> `Push` -> `Docker Hub`.

Tip de Gestión del Aula:

- Indicar a los alumnos que abran su editor de código (VS Code) y su terminal.

Diapositiva 3: Objetivo de la Sesión 2

Contenido de la PPT:

Objetivo de la sesión 02:

- Crear imágenes profesionales de aplicaciones propias.
- Entender el funcionamiento de las capas y la caché de build.
- Aplicar buenas prácticas para reducir el tamaño de las imágenes.
- Usar `.dockerignore` para excluir archivos innecesarios.
- Versionalizar imágenes con tags claros.
- Publicar y descargar imágenes desde Docker Hub.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Nuestro **Objetivo de la Sesión 2** es convertirnos en arquitectos de imágenes. Al finalizar esta clase, cada uno de ustedes será capaz de redactar un Dockerfile optimizado, entenderá por qué el orden de las líneas afecta la velocidad de compilación, sabrá reducir el peso de una imagen de 900MB a menos de 50MB y sabrá subir y descargar sus propias imágenes desde Docker Hub."

🏠 Acción en Consola / Pizarra:

- Resaltar la palabra **OPTIMIZACIÓN**: Pasar de imágenes pesadas y lentas a imágenes livianas y reproducibles.

💡 Tip de Gestión del Aula:

- Motivar a los alumnos: *"Hoy aprenderán las mismas prácticas de compilación que usan empresas como Netflix, Google o AWS en producción"*.

📄 Diapositiva 4: Bloque 1 — Anatomía del Dockerfile

Contenido de la PPT:

Anatomía del Dockerfile

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Iniciamos con el **Bloque 1: Anatomía del Dockerfile**. Vamos a entender qué es este archivo y por qué constituye la base de la *Infraestructura como Código (IaC)*."

🏠 Acción en Consola / Pizarra:

- Escribir en la pizarra: **Dockerfile = Receta estandarizada e inmutable de la Imagen.**

💡 Tip de Gestión del Aula:

- Recordar que el archivo se debe llamar exactamente **Dockerfile** (sin extensión **.txt** ni letras mayúsculas intermedias).

📄 Diapositiva 5: Lo que ya dominamos / Base para hoy

Contenido de la PPT:

Lo que ya dominamos:
Sabemos ejecutar contenedores desde imágenes existentes.
Ahora crearemos nuestras propias imágenes de forma profesional.
Base para hoy.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Repasemos lo aprendido: En la Sesión 1 descargamos imágenes preexistentes (`nginx`, `ubuntu`, `hello-world`) y las ejecutamos. Pero en un entorno de trabajo real, las empresas no usan la imagen por defecto sin modificar; necesitan empaquetar SU propio código Python, Node, Java o Go con SUS dependencias. Hoy aprenderemos a crear esas imágenes personalizadas."

👤 Acción en Consola / Pizarra:

- Mostrar la evolución: `Sesión 1: docker run nginx` (Imagen de otros) -> `Sesión 2: docker build` (Imagen PROPIA).

💡 Tip de Gestión del Aula:

- Verificar que todos tengan descargada la carpeta `codigo/sesion2` del repositorio.

📄 Diapositiva 6: ¿Qué es un Dockerfile?

Contenido de la PPT:

¿QUÉ ES UN DOCKERFILE?

DEFINICIÓN:

Un Dockerfile es un archivo de texto con instrucciones que Docker lee para construir una imagen automáticamente.

Es la receta que define todo lo que necesita tu aplicación para ejecutarse.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"La **Definición Oficial**: Un **Dockerfile** es un archivo de texto plano sin extensión que contiene una serie de instrucciones declarativas. El motor de Docker lee este archivo en orden secuencial para compilar automáticamente una imagen. Es la 'receta de cocina' que define el sistema operativo base, los programas a instalar, los archivos a copiar y el comando final de inicio."

👤 Acción en Consola / Pizarra:

- Dibujar el flujo: `[ Dockerfile ] --(docker build)--> [ Imagen ] --(docker run)--> [ Contenedor ]`.

💡 Tip de Gestión del Aula:

- Resaltar que por convención el archivo se ubica en la raíz del proyecto fuente.

📄 Diapositiva 7: Bloque 2 — Instrucciones clave del Dockerfile

Contenido de la PPT:

Instrucciones clave del Dockerfile

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Entramos al **Bloque 2: Instrucciones clave del Dockerfile**. Vamos a desglosar una a una las sentencias reservadas que componen cualquier receta Docker."

🏠 Acción en Consola / Pizarra:

- Listar las palabras clave en la pizarra: `FROM`, `RUN`, `COPY`, `ADD`, `WORKDIR`, `ENV`, `EXPOSE`, `CMD`, `ENTRYPOINT`.

💡 Tip de Gestión del Aula:

- Pedir a los alumnos que presten especial atención a las diferencias entre `COPY` vs `ADD` y `CMD` vs `ENTRYPOINT`.

📄 Diapositiva 8: FROM: El punto de partida

Contenido de la PPT:

FROM: EL PUNTO DE PARTIDA

- Define la imagen base sobre la que se construye.
- Es la primera instrucción de todo Dockerfile.
- Puede ser una imagen oficial (python, node, nginx) o una imagen propia.

CONSEJO: Usa imágenes oficiales siempre que puedas. Son mantenidas, actualizadas y más seguras.

VERSIONES: Evita `:latest` en producción. Fija una versión concreta (`:3.12-slim`) para builds reproducibles.

Ejemplo:

```
FROM python:3.12-slim
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"La primera instrucción de TODO Dockerfile es **FROM**. `FROM` define la **imagen base** sobre la cual construiremos nuestra aplicación. Ningún Dockerfile puede empezar sin un `FROM`. **Consejos de Producción:**

- Usen siempre imágenes oficiales de Docker Hub (tienen el sello de verificación).
- NUNCA usen `:latest` en producción.** El tag `:latest` cambia con el tiempo y romperá la reproducibilidad de sus compilaciones. Fijen siempre la versión exacta, por ejemplo `python:3.12-slim`."

🏠 Acción en Consola / Pizarra:

```
# Mal para producción:
```

```
FROM python:latest
```

```
# Bien (Profesional y reproducible):
```

```
FROM python:3.12-slim
```

💡 Tip de Gestión del Aula:

- Explicar qué significa **slim**: una variante oficial reducida de Debian que no incluye compiladores innecesarios, ocupando ~150MB en lugar de 900MB.

📄 Diapositiva 9: RUN: Ejecutar comandos durante el build

Contenido de la PPT:

RUN: EJECUTAR COMANDOS DURANTE EL BUILD

- Ejecuta comandos durante la construcción de la imagen.
- Se usa para instalar paquetes, crear carpetas, configurar el sistema.
- Cada RUN crea una nueva capa en la imagen.

BUENA PRÁCTICA: Encadena comandos con `&&` para reducir capas y limpiar cachés en la misma instrucción.

REGLA DE ORO: Limpia archivos temporales (`apt lists`, `pip cache`) en el mismo RUN para no arrastrar basura a la imagen final.

Ejemplo:

```
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"La instrucción **RUN** se utiliza para ejecutar comandos de consola **DURANTE la construcción (build) de la imagen**. Se usa para instalar software (`apt-get`, `pip`, `npm`), crear directorios o modificar permisos. **Punto crítico:** Cada instrucción **RUN** crea una capa física e inmutable en el disco. **Regla de Oro:** Encadenen múltiples comandos usando `&&` \ y limpien archivos temporales en el MISMO bloque **RUN**. Si borran la caché de instalación en una línea **RUN** posterior, el archivo ya habrá quedado grabado en la capa anterior y la imagen seguirá pesando lo mismo."

🖥️ Acción en Consola / Pizarra:

Mal (Genera 3 capas pesadas):

```
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*
```

Bien (Genera 1 sola capa limpia):

```
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```

💡 Tip de Gestión del Aula:

- Enfatizar la diferencia: **RUN** se ejecuta al construir la imagen (**docker build**). No confundir con comandos al iniciar el contenedor.

Diapositiva 10: COPY vs ADD

Contenido de la PPT:

COPY VS ADD

COPY: Copia archivos o carpetas desde el contexto de build (tu PC) hacia la imagen.

Es la opción recomendada para la mayoría de casos.

Ejemplo:

```
COPY requirements.txt .
```

```
COPY app.py /app/
```

ADD: Además de copiar, desprime automáticamente archivos **.tar.gz** y puede descargar desde URLs.

Úsalo solo cuando necesites esa funcionalidad extra.

Ejemplo:

```
ADD app.tar.gz /app/
```

```
ADD https://example.com/file /tmp/
```

REGLA PRÁCTICA: prefiere **COPY**. Usa **ADD** solo si necesitas descomprimir automáticamente.

Guión del Docente (Lo que debes decir a los alumnos):

"Analicemos dos instrucciones que suelen confundir a los principiantes: **COPY** vs **ADD**.

- **COPY:** Es la instrucción limpia y transparente. Copia archivos o carpetas de tu laptop física hacia el sistema de archivos de la imagen. Es el 95% de los casos que usarán.
- **ADD:** Es una instrucción con superpoderes. Además de copiar, **descomprime automáticamente** archivos comprimidos (como **.tar.gz**) y puede descargar archivos desde URLs de internet. **Regla Práctica:** Usen siempre **COPY** por seguridad y transparencia. Usen **ADD** únicamente cuando requieran descomprimir un tarball al instante."

Acción en Consola / Pizarra:

- Escribir la regla en la pizarra: **COPY = Copia simple (Recomendado) | ADD = Copia + Descompresión automática / URL.**

Tip de Gestión del Aula:

- Advertir que usar **ADD** para descargar archivos remotos no es recomendado porque no limpia la caché de descarga.

Diapositiva 11: WORKDIR: El directorio de trabajo

Contenido de la PPT:

WORKDIR: EL DIRECTORIO DE TRABAJO

- Establece el directorio de trabajo para las instrucciones siguientes.
- Si la carpeta no existe, Docker la crea automáticamente.
- Afecta a RUN, CMD, ENTRYPOINT, COPY y ADD.

¿POR QUÉ USARLO?

Evita rutas absolutas en cada comando.

Hace el Dockerfile más legible.

Previene errores de archivos en carpetas equivocadas.

PREFIERE WORKDIR: Prefiere WORKDIR /app; RUN cd ... no persiste entre instrucciones.

Ejemplo:

```
WORKDIR /app
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"La instrucción **WORKDIR** establece la carpeta de trabajo interna dentro de la imagen. Si la carpeta especificada no existe, Docker la creará automáticamente. **Cuidado común:** Un error típico de principiante es usar **RUN cd /app**. En un Dockerfile, cada instrucción **RUN** se ejecuta en un subshell independiente; por lo tanto, un **RUN cd** NO se mantiene para las siguientes líneas. Usen siempre **WORKDIR /app**. A partir de esa línea, todas las instrucciones relativas (como **COPY . .**) operarán dentro de **/app**."

🏠 Acción en Consola / Pizarra:

```
# Mal (no persiste el directorio):  
RUN mkdir /app  
RUN cd /app  
COPY app.py .    # ✗ Se copia en la raíz / en lugar de /app  
  
# Bien (Profesional):  
WORKDIR /app  
COPY app.py .    # Se copia correctamente en /app/app.py
```

💡 Tip de Gestión del Aula:

- Mostrar cómo **WORKDIR /app** limpia el código evitando escribir **/app/** en cada instrucción.

📄 Diapositiva 12: ENV: Variables de entorno**Contenido de la PPT:****ENV: VARIABLES DE ENTORNO**

- Define variables de entorno disponibles en tiempo de build y en el contenedor.
- Ideal para configuraciones que no cambian entre entornos.

- Se pueden sobrescribir con `docker run -e`.

CUÁNDO USAR ENV: Configuración por defecto de la app, versiones de herramientas, rutas y paths estándar.

CUÁNDO NO: Secretos como contraseñas o tokens NO deben quedar hardcoded en la imagen.

Ejemplo:

```
ENV FLASK_ENV=production
```

```
ENV APP_HOME=/app
```

```
ENV PYTHONUNBUFFERED=1
```

Guión del Docente (Lo que debes decir a los alumnos):

"La instrucción **ENV** define variables de entorno persistentes dentro de la imagen. Estas variables quedan grabadas y estarán disponibles tanto durante la construcción de la imagen como cuando el contenedor se ejecute. **Cuándo usar ENV:** Para banderas de configuración estándar, como `PYTHONUNBUFFERED=1` (para ver los logs en tiempo real sin búfer) o `FLASK_ENV=production`. **Cuándo NUNCA usar ENV en Dockerfile: Jamás dejen claves API, secretos de JWT o contraseñas de BD quemadas con ENV.** Cualquier persona que descargue la imagen puede ejecutar `docker inspect` y leer sus secretos en texto plano."

Acción en Consola / Pizarra:

- Advertencia en la pizarra en rojo: ¡SECRETO EN ENV = VULNERABILIDAD GRAVE!.

Tip de Gestión del Aula:

- Explicar que las variables `ENV` pueden sobrescribirse al ejecutar el contenedor con `docker run -e CLAVE=VALOR`.

Diapositiva 13: EXPOSE: Declarar puertos

Contenido de la PPT:

EXPOSE: DECLARAR PUERTOS

- Documenta qué puertos escucha el contenedor.
- No publica el puerto automáticamente.
- Sirve como metadata para otros desarrolladores y herramientas.

EXPOSE VS -P:

EXPOSE: declara (informativo).

`docker run -p`: publica y mapea el puerto.

Ejemplo:

```
EXPOSE 5000
```

```
EXPOSE 80/tcp
```

Guión del Docente (Lo que debes decir a los alumnos):

"La instrucción **EXPOSE** tiene una función puramente **documental e informativa**. **EXPOSE 5000** le dice a otros desarrolladores y herramientas que la aplicación dentro de la imagen escucha en el puerto 5000. **Atención:** **EXPOSE** por sí solo **NO abre ni publica el puerto hacia tu máquina física**. Para abrir y redirigir el puerto hacia tu laptop, sigues necesitando usar el flag **-p** al ejecutar: **docker run -p 5000:5000**."

Acción en Consola / Pizarra:

- Comparación rápida:
 - **EXPOSE 5000** -> Comentario de documentación técnica dentro de la receta.
 - **docker run -p 5000:5000** -> Apertura real del cable de red virtual.

Tip de Gestión del Aula:

- Preguntar a la clase si un contenedor responde sin el flag **-p** aunque tenga **EXPOSE** en su Dockerfile. (Respuesta: No responde externamente).

Diapositiva 14: CMD vs ENTRYPOINT

Contenido de la PPT:

CMD VS ENTRYPOINT

CMD:

- Comando por defecto al iniciar el contenedor.
- Se puede sobrescribir fácilmente desde **docker run**.
- Forma **exec** (array) recomendada sobre forma **shell**.

Ejemplo:

```
CMD ["python", "app.py"]
```

ENTRYPOINT:

- Define el ejecutable principal (no se sobrescribe fácilmente).
- CMD proporciona argumentos por defecto al ENTRYPOINT.
- Útil para crear herramientas CLI en contenedores.

Ejemplo:

```
ENTRYPOINT ["python"]
```

```
CMD ["app.py"]
```

REGLA GENERAL: Usa CMD para apps. Usa ENTRYPOINT + CMD para herramientas CLI.

Guión del Docente (Lo que debes decir a los alumnos):

"Llegamos a un tema clave de arquitectura: **CMD vs ENTRYPOINT**.

- **CMD:** Define el proceso y argumentos por defecto al arrancar. Si el usuario pasa un comando al final de **docker run** (ej. **docker run mi-imagen bash**), ese comando sobrescribe por completo el **CMD**.
- **ENTRYPOINT:** Convierte al contenedor en un ejecutable fijo. Los argumentos pasados en **docker run** no reemplazan al ejecutable, sino que se le pasan como parámetros.

- **Combinación avanzada:** Puedes usar `ENTRYPOINT ["python"]` y `CMD ["app.py"]`. Así, el binario fijo es Python y el argumento por defecto es `app.py`. Si ejecutas `docker run mi-imagen test.py`, ejecutará `python test.py`. **Regla General:** Para aplicaciones web estándar usar `CMD ["python", "app.py"]` en **Forma Exec (con corchetes [])**."

Acción en Consola / Pizarra:

- Explicar las dos formas sintácticas:
 - *Forma Exec (Recomendada):* `CMD ["python", "app.py"]` -> Ejecuta el proceso directamente como PID 1 (recibe señales de apagado SIGTERM).
 - *Forma Shell (Evitar):* `CMD python app.py` -> Ejecuta `/bin/sh -c`, impidiendo que el proceso reciba señales de parada limpias.

Tip de Gestión del Aula:

- Resaltar por qué la forma Exec con lista `["binario", "arg"]` es el estándar en producción.

Diapositiva 15: Resumen visual de instrucciones

Contenido de la PPT:

RESUMEN VISUAL DE INSTRUCCIONES

INSTRUCCIÓN	¿QUÉ HACE?	SOLUCIÓN RÁPIDA / USO
FROM	Define la imagen base	Siempre como primera línea
RUN	Ejecuta comandos en el build	Instalar dependencias, scripts
COPY	Copia archivos locales a la imagen	Mover código y archivos del proyecto
WORKDIR	Establece el directorio de trabajo	Al inicio y al cambiar contexto
ENV	Define variables de entorno	Configuraciones por defecto
EXPOSE	Declara puertos de escucha	Documentar qué puerto usa la app
CMD	Comando por defecto del contenedor	Iniciar la aplicación

Guión del Docente (Lo que debes decir a los alumnos):

"Aquí tienen la tabla del **Resumen Visual de Instrucciones**. Revisen la secuencia lógica típica de un Dockerfile profesional:

1. **FROM** -> Elegir la imagen base.
2. **WORKDIR** -> Definir la carpeta interna.
3. **COPY** -> Copiar requisitos.
4. **RUN** -> Instalar dependencias.
5. **COPY** -> Copiar el código fuente.
6. **EXPOSE** -> Documentar el puerto.
7. **CMD** -> Definir la instrucción de arranque."

Acción en Consola / Pizarra:

- Proyectar la tabla durante 1 minuto como chuleta de referencia rápida.

💡 Tip de Gestión del Aula:

- Preguntar si los alumnos ven clara la secuencia lógica de los 7 pasos antes de entrar al tema de capas y caché.

📄 Diapositiva 16: Bloque 3 — Capas, caché y buenas prácticas

Contenido de la PPT:

Capas, caché y buenas prácticas

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Ingresamos al **Bloque 3: Capas, caché y buenas prácticas**. Entender cómo Docker compila las imágenes internamente marcará la diferencia entre un desarrollador novato y un profesional DevOps."

🏠 Acción en Consola / Pizarra:

- Dibujar un bloque de capas apiladas en la pizarra.

💡 Tip de Gestión del Aula:

- Anunciar que comprender la caché les ahorrará horas de espera en sus pipelines de CI/CD.

📄 Diapositiva 17: ¿Cómo construye Docker una imagen?

Contenido de la PPT:

¿CÓMO CONSTRUYE DOCKER UNA IMAGEN?

- Cada instrucción del Dockerfile genera una capa.
- Las capas son inmutables.
- Docker reutiliza las que no cambiaron mediante caché.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Analicemos el motor interno: ¿Cómo construye Docker una imagen?"

1. Cada instrucción en un Dockerfile (como **FROM**, **RUN**, **COPY**) genera una **capa de sistema de archivos (Layer)**.
2. Todas las capas son **inmutables** (de solo lectura) y se apilan una sobre otra usando un sistema de archivos especial (UnionFS).
3. Cuando recompilas la imagen con **docker build**, Docker revisa si la instrucción y los archivos copiados en esa capa han sufrido algún cambio. Si no cambiaron, **reutiliza la capa desde el caché local en 0 segundos**."

🏠 Acción en Consola / Pizarra:

- Ilustrar las capas apiladas: [Capa 4: CMD] -> [Capa 3: COPY app.py] -> [Capa 2: RUN pip install] -> [Capa 1: FROM python]

💡 Tip de Gestión del Aula:

- Mostrar cómo el hashing del contenido detecta cambios en los archivos copiados.

📄 Diapositiva 18: La caché de Docker: cómo funciona

Contenido de la PPT:

LA CACHÉ DE DOCKER: CÓMO FUNCIONA

- Docker compara cada instrucción con la caché de builds anteriores.
- Si la instrucción y sus dependencias no cambiaron, reutiliza la capa.
- Si una capa cambia, todas las capas siguientes se reconstruyen.

REGLA DE ORO DEL ORDEN: Pon primero lo que cambia menos (instalación de dependencias) y al final lo que cambia más (código de la app).

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Esta es la **Regla de Oro del Orden de Capas**: Si una sola capa cambia (por ejemplo, editaste una línea en `app.py`), **Docker invalida la caché de esa capa Y DE TODAS LAS CAPAS SIGUIENTES**, obligando a reconstruirlas desde cero. Por lo tanto, la regla de arquitectura es: **Coloca al principio del Dockerfile lo que cambia RARA VEZ (instalación de dependencias y paquetes de SO), y coloca al final de la receta lo que cambia CONSTANTEMENTE (el código fuente de tu aplicación).**"

🏠 Acción en Consola / Pizarra:

- Subrayar en la pizarra: **Lo estable PRIMERO, lo frecuente AL FINAL.**

💡 Tip de Gestión del Aula:

- Preguntar: "Si coloco `COPY . .` al principio de mi Dockerfile y edito `app.py`, ¿qué pasará con el `RUN pip install` que está después?" (Respuesta: Se romperá la caché y reinstalará todas las librerías desde internet perdiendo minutos).

📄 Diapositiva 19: Orden importa: Ejemplo práctico

Contenido de la PPT:

ORDEN IMPORTA: EJEMPLO PRÁCTICO

Mal orden (lento):

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY . . # ✗ Cambia siempre (invalida la caché del RUN)
4 RUN pip install -r requirements.txt
```

```
5 CMD ["python", "app.py"]
```

Buen orden (rápido):

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt . # Cambia raras veces (aprovecha la caché)
4 RUN pip install -r requirements.txt
5 COPY . . # Código al final
6 CMD ["python", "app.py"]
```

Guión del Docente (Lo que debes decir a los alumnos):

"Comparemos los dos ejemplos en pantalla:

- **En el MAL orden:** Copias todo el código (**COPY . .**) ANTES de instalar dependencias. Cada vez que edites un comentario en **app.py**, Docker invalidará la caché del **COPY . .**, forzando a ejecutar **RUN pip install** e instalando todas las librerías de internet de nuevo. Tiempo de build: 45 segundos.
- **En el BUEN orden:** Copias SOLO **requirements.txt**, ejecutas **RUN pip install**, y LUEGO copias el código fuente (**COPY . .**). Si editas **app.py**, Docker reutiliza la caché de **pip install** y el build tarda **0.5 segundos**."

Acción en Consola / Pizarra:

- Demostrar el concepto en la terminal durante el laboratorio para que vean la palabra **---> Using cache** en pantalla.

Tip de Gestión del Aula:

- Ver si los estudiantes identificaron por qué el segundo orden es la práctica estándar en la industria.

Diapositiva 20: Otras buenas prácticas

Contenido de la PPT:

OTRAS BUENAS PRÁCTICAS

1. Usa imágenes base específicas:
FROM python:3.12-slim (evita FROM python genérico)
2. No instales paquetes innecesarios:
RUN apt-get install -y --no-install-recommends curl
3. Limpia en el mismo RUN:
RUN pip install -r requirements.txt && rm -rf /root/.cache/pip
4. Usa multistage builds:
Separa la etapa de build de la imagen de runtime.

Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos estas 4 buenas prácticas profesionales para el examen y su trabajo diario:

1. **Fijen la versión exacta:** Usen `python:3.12-slim` en vez de `python` genérico.
2. **Eviten paquetes basura:** Usen `--no-install-recommends` en `apt-get` para no descargar utilidades recomendadas innecesarias.
3. **Limpiesen cachés en la misma instrucción:** Agreguen `rm -rf /root/.cache/pip` o `--no-cache-dir` al instalar paquetes Python.
4. **Implementen Multi-stage builds:** Separen la compilación pesada de la ejecución final."

Acción en Consola / Pizarra:

- Anotar los 4 mandamientos de optimización en la esquina de la pizarra.

Tip de Gestión del Aula:

- Introducir el concepto del siguiente slide: ¿Qué es una compilación Multi-stage?

Diapositiva 21: Multistage Build: El patrón profesional

Contenido de la PPT:

MULTISTAGE BUILD: EL PATRÓN PROFESIONAL

VENTAJA:

La imagen final solo contiene lo necesario para ejecutar. Las herramientas de compilación quedan en la etapa de build y no contaminan la imagen de producción.

Ejemplo:

```
# Stage 1: build
```

```
FROM golang:1.21 AS builder
```

```
RUN go build -o app
```

```
# Stage 2: runtime (imagen final pequeña)
```

```
FROM alpine:3.19
```

```
COPY --from=builder /app .
```

```
CMD ["/app"]
```

Guión del Docente (Lo que debes decir a los alumnos):

"El patrón **Multi-stage build** es la técnica más avanzada de optimización de imágenes. Permite usar múltiples instrucciones `FROM` en un solo Dockerfile:

- **Etapas 1 (builder):** Usa una imagen pesada con compiladores (GCC, Go, SDKs) para construir la aplicación.
- **Etapas 2 (runtime):** Parte de una imagen ultraligera (como `alpine` o `slim`) y copia ÚNICAMENTE el ejecutable binario resultante mediante `COPY --from=builder`. **Resultado:** La imagen final pesa solo 15MB en lugar de 1GB y queda libre de herramientas de compilación que un atacante podría usar."

Acción en Consola / Pizarra:

- Dibujar el embudo Multi-stage: [Stage 1: Builder (1 GB)] --(compila app)--> COPY --from=builder --> [Stage 2: Runtime Final (20 MB)]

💡 Tip de Gestión del Aula:

- Destacar que este patrón reduce drásticamente las vulnerabilidades (CVEs) reportadas en escaneos de seguridad.

📄 Diapositiva 22: Bloque 4 — .dockerignore e imágenes base livianas

Contenido de la PPT:

.dockerignore e imágenes base livianas

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Entramos al **Bloque 4: .dockerignore e imágenes base livianas**. Vamos a aprender cómo evitar enviar gigabytes de datos basura al demonio de Docker y cómo elegir la distribución base correcta."

👤 Acción en Consola / Pizarra:

- Escribir en la pizarra: `.dockerignore` = El filtro de seguridad y velocidad del build.

💡 Tip de Gestión del Aula:

- Preguntar cuántos conocen el funcionamiento del archivo `.gitignore`.

📄 Diapositiva 23: .dockerignore: Qué es y para qué sirve

Contenido de la PPT:

.DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE

REGLA: Incluye .dockerignore; evita enviar node_modules, .git y credenciales.

- Similar a .gitignore: define archivos y carpetas que NO se envían al contexto de build.
- Reduce el tamaño del contexto enviando menos MBs al demonio.
- Acelera el inicio del build.
- Evita que archivos sensibles (.env, claves, logs) terminen copiados dentro de la imagen.

Ejemplo de .dockerignore:

```
__pycache__
*.pyc
.git
.env
venv/
node_modules
*.md
```


👤 Guión del Docente (Lo que debes decir a los alumnos):

"El archivo `.dockerignore` funciona de manera idéntica a `.gitignore`. Cuando ejecutas `docker build .`, la consola empaqueta todo el contenido de la carpeta y se lo envía al motor demonio de Docker (el llamado *Build Context*). Si tienes carpetas pesadas como `node_modules`, entornos virtuales `venv/` o el historial de `.git`, enviarás cientos de megabytes innecesarios antes de empezar a compilar. Peor aún: si no usas `.dockerignore`, tu archivo `.env` con contraseñas locales terminará dentro de la imagen. **Regla obligatoria:** Creen siempre un `.dockerignore` incluyendo `__pycache__`, `.git`, `.env`, `venv/` y `node_modules`."

🏠 Acción en Consola / Pizarra:

- Mostrar en la terminal el mensaje `Sending build context to Docker daemon 2.5MB vs 250MB`.

💡 Tip de Gestión del Aula:

- Señalar que un `.dockerignore` adecuado puede hacer que la compilación empiece al instante.

📄 Diapositiva 24: Imágenes base: Slim vs Alpine

Contenido de la PPT:

IMÁGENES BASE: SLIM VS ALPINE

¿CUÁL ELEGIR?

- Slim:** buena relación tamaño/compatibilidad (~150MB).
- Alpine:** máxima reducción (~5MB), pero puede tener problemas con paquetes que dependen de `glibc` (usa `musl C`).
- Para empezar y en la mayoría de proyectos Python/Node, `slim` es la opción más segura.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Comparemos las dos variantes de imágenes base más populares:

- Variante Slim (Debian Slim):** Pesa aproximadamente 150MB. Mantiene la biblioteca estándar de C (`glibc`). Ofrece la mejor compatibilidad con librerías compiladas como `psycpg2`, `pandas` o `numpy`.
- Variante Alpine (Alpine Linux):** Pesa solo 5MB. Es ultra liviana, pero utiliza una biblioteca C diferente llamada `musl`. Algunos paquetes de Python o C++ pueden fallar o requerir compilación lenta desde cero. **Recomendación profesional:** Para iniciarse y para la mayoría de entornos de producciones reales en Python/Node, **`slim` es la opción recomendada y más segura.**

🏠 Acción en Consola / Pizarra:

- Escribir la comparativa:
 - `python:3.12-slim` -> 150MB | Usa `glibc` | Máxima compatibilidad (RECOMENDADO).

- `python:3.12-alpine` -> 50MB | Usa `musl` | Requiere validar librerías C.

💡 Tip de Gestión del Aula:

- Preguntar si entendieron por qué Alpine a veces falla al instalar librerías con Wheels binarios de pip.

📄 Diapositiva 25: Bloque 5 — Publicación en Docker Hub

Contenido de la PPT:

Publicación en Docker Hub

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Entramos al **Bloque 5: Publicación en Docker Hub**. Ya sabemos crear nuestras imágenes locales; ahora aprenderemos a publicarlas en la nube para que nuestros compañeros o servidores puedan descargarlas."

🏠 Acción en Consola / Pizarra:

- Mostrar en pantalla el sitio web `hub.docker.com`.

💡 Tip de Gestión del Aula:

- Pedir a los alumnos que tengan sus cuentas de Docker Hub abiertas.

📄 Diapositiva 26: Flujo de publicación

Contenido de la PPT:

FLUJO DE PUBLICACIÓN
EL CICLO COMPLETO:
1. Construyes tu imagen localmente.
2. Le asignas un tag con el formato `usuario/imagen:tag`.
3. La publicas en el registro con `docker push`.
4. Cualquiera puede descargarla con `docker pull` y usarla.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"El **Flujo de Publicación** se compone de 4 pasos continuos:

1. **Build:** Construyes tu imagen local con `docker build`.
2. **Tag:** Le asignas una etiqueta oficial con tu nombre de usuario de Docker Hub:
`usuario/imagen:tag`.
3. **Push:** Subes la imagen al registro remoto con `docker push`.
4. **Pull/Run:** Cualquier servidor o compañero puede descargarla con `docker pull` y ejecutarla con `docker run`."

Acción en Consola / Pizarra:

- Dibujar el ciclo en la pizarra: `PC Local [Build] -> [Tag] -> [Push] ---> ( Docker Hub Cloud ) ---> [Pull/Run] -> Servidor Producción`

Tip de Gestión del Aula:

- Comparar este flujo con `git commit` y `git push` en GitHub para que los desarrolladores lo asocien de inmediato.

Diapositiva 27: Login y Tagging

Contenido de la PPT:

LOGIN Y TAGGING

1. Iniciar sesión en Docker Hub:

```
docker login
```

```
# Usuario: tu-usuario
```

```
# Password: tu-token / contraseña
```

2. Etiquetar la imagen:

```
# Formato: usuario/imagen:tag
```

```
docker tag mi-flask:v1 tu-usuario/mi-flask:v1
```

CONVENCIÓN DE TAGS:

- `v1`, `v2`, `v3`: versiones específicas.
- `latest`: la más reciente (se actualiza manualmente).
- `dev`, `staging`, `prod`: por entorno.
- `2026-05-31`: por fecha de build.

IMPORTANTE: `latest` NO es automático: tú decides qué imagen lo lleva.

Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos los dos primeros comandos de publicación:

1. **docker login**: Autentica tu consola contra Docker Hub. Ingresas tu usuario y tu token/contraseña.
2. **docker tag**: Crea un alias o puntero de tu imagen local asignándole el formato de nombre remoto obligatorio: `usuario/nombre-imagen:etiqueta`.

Convención de Tags: Usen etiquetas semánticas (`v1`, `v1.0.2`), etiquetas de entorno (`prod`, `staging`) o fechas. **Mito a romper:** El tag `latest` **NO se asigna automáticamente al hacer build**. Es simplemente un nombre de texto común. Tú debes asignarlo explícitamente si deseas etiquetar una imagen como `latest`."

Acción en Consola / Pizarra:

```
# Ejemplo práctico de tagging:
docker tag mi-flask:v2 cristian/mi-flask:v2
docker tag mi-flask:v2 cristian/mi-flask:latest
```

💡 Tip de Gestión del Aula:

- Advertir a los estudiantes que para publicar en Docker Hub, el nombre de la imagen DEBE comenzar obligatoriamente con su nombre de usuario de Docker Hub.

📄 Diapositiva 28: Push y Pull

Contenido de la PPT:

PUSH Y PULL

3. Publicar la imagen:

```
docker push tu-usuario/mi-flask:v1
docker push tu-usuario/mi-flask:latest
```

4. Descargar y ejecutar en otro equipo:

```
docker pull tu-usuario/mi-flask:v1
docker run -d -p 5000:5000 tu-usuario/mi-flask:v1
```

¿PÚBLICA O PRIVADA?

Docker Hub permite 1 repositorio privado gratis.

GHCR (GitHub Container Registry) es la alternativa recomendada para repos privados ilimitados.

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Completamos la publicación con **docker push**: Al ejecutar **docker push tu-usuario/mi-flask:v1**, las capas de tu imagen se suben de forma comprimida a la nube de Docker Hub. Si una capa ya existe en Docker Hub (por ejemplo, la base de Python), Docker NO la vuelve a subir, ahorrando ancho de banda.

Luego, en cualquier servidor del mundo, cualquier persona podrá ejecutar: **docker run -d -p 5000:5000 tu-usuario/mi-flask:v1** y la app levantará exactamente igual. **Sobre visibilidad:**

Docker Hub ofrece 1 repositorio privado gratuito. Si requieren repositorios privados ilimitados, utilizaremos GHCR (GitHub Container Registry) como veremos en la sesión 5."

🏠 Acción en Consola / Pizarra:

```
docker push cristian/mi-flask:v2
# En otra computadora o consola:
docker run -d -p 5000:5000 cristian/mi-flask:v2
```

💡 Tip de Gestión del Aula:

- Mostrar cómo Docker Hub indica en pantalla que las capas comunes dicen **Layer already exists**.
-

Diapositiva 29: Bloque 6 — Laboratorio práctico

Contenido de la PPT:

Laboratorio práctico

Guión del Docente (Lo que debes decir a los alumnos):

"Ingresamos al **Bloque 6: Laboratorio práctico**. Vamos a consolidar todo lo aprendido en una secuencia práctica paso a paso directamente en la consola."

Acción en Consola / Pizarra:

- Abrir la terminal y preparar el directorio del laboratorio.

Tip de Gestión del Aula:

- Verificar que todos los alumnos sigan los comandos al mismo tiempo.
-

Diapositiva 30: Laboratorio: App Flask Optimizada

Contenido de la PPT:

LABORATORIO: APP FLASK OPTIMIZADA

OBJETIVO:

Partiendo del proyecto Flask de la sesión 1, optimizar el Dockerfile con buenas prácticas, construir la imagen aprovechando la caché y publicarla en Docker Hub.

Guión del Docente (Lo que debes decir a los alumnos):

"Nuestro **Objetivo del Laboratorio**: Vamos a tomar la app Flask, redactaremos un Dockerfile optimizado usando el patrón Multi-stage, compilaremos la imagen **mi-flask:v2**, verificaremos sus capas con **docker history** y la publicaremos exitosamente en Docker Hub."

Acción en Consola / Pizarra:

- Mostrar la estructura de la carpeta **codigo/sesion2** en el VS Code.

Tip de Gestión del Aula:

- Guiar a los estudiantes para que abran el archivo **Dockerfile.multistage**.
-

Diapositiva 31: Dockerfile optimizado para el laboratorio

Contenido de la PPT:

```
DOCKERFILE OPTIMIZADO PARA EL LABORATORIO
# Etapa 1: builder
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Etapa 2: runtime (imagen final)
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.12/site-packages
/usr/local/lib/python3.12/site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Guión del Docente (Lo que debes decir a los alumnos):

"Revisemos el Dockerfile del laboratorio:

- **Etapa 1 (builder):** Parte de `python:3.12-slim`, instala todas las librerías necesarias con `pip install --no-cache-dir`.
- **Etapa 2 (runtime):** Parte de una base limpia, copia las librerías compiladas desde la etapa previa (`COPY --from=builder ...`), copia nuestro `app.py` y define el comando de inicio `CMD ["python", "app.py"]`. De esta forma logramos un build limpio, ligero e inmutable."

Acción en Consola / Pizarra:

- Explicar la línea `COPY --from=builder`: la magia que permite traer archivos seleccionados de una etapa anterior descartando la basura del builder.

Tip de Gestión del Aula:

- Asegurarse de que los alumnos no tengan errores tipográficos en las rutas de site-packages.

Diapositiva 32: Comandos del laboratorio

Contenido de la PPT:

```
COMANDOS DEL LABORATORIO
# 1. Construir la imagen
docker build -t mi-flask:v2 .

# 2. Ver las capas generadas
docker history mi-flask:v2

# 3. Ejecutar y probar
docker run -d --name flask-v2 -p 5000:5000 mi-flask:v2
```

```
curl http://localhost:5000

# 4. Login en Docker Hub
docker login

# 5. Etiquetar
docker tag mi-flask:v2 tu-usuario/mi-flask:v2

# 6. Publicar
docker push tu-usuario/mi-flask:v2

# 7. Verificar en: hub.docker.com/r/tu-usuario/mi-flask
```

Guión del Docente (Lo que debes decir a los alumnos):

"Ejecutemos la secuencia de 7 pasos del laboratorio:

1. Compilamos: `docker build -t mi-flask:v2 .`
2. Inspeccionamos sus capas: `docker history mi-flask:v2`.
3. Probamos localmente: `docker run -d --name flask-v2 -p 5000:5000 mi-flask:v2` y probamos con `curl http://localhost:5000`.
4. Autenticamos: `docker login`.
5. Etiquetamos: `docker tag mi-flask:v2 tu-usuario/mi-flask:v2`.
6. Publicamos: `docker push tu-usuario/mi-flask:v2`.
7. Abran su navegador y verifiquen que su repositorio en `hub.docker.com` ya muestra la versión **v2** subida."

Acción en Consola / Pizarra:

```
docker build -t mi-flask:v2 .
docker history mi-flask:v2
docker run -d --name flask-v2 -p 5000:5000 mi-flask:v2
docker login
docker tag mi-flask:v2 cristian/mi-flask:v2
docker push cristian/mi-flask:v2
```

Tip de Gestión del Aula:

- Celebrar cuando los alumnos confirmen que ven su repositorio publicado en Docker Hub.

Diapositiva 33: Errores frecuentes en esta sesión

Contenido de la PPT:

```
ERRORES FRECUENTES EN ESTA SESIÓN
| ERROR | CAUSA PROBABLE | SOLUCIÓN RÁPIDA |
|---|---|---|
| Build lento siempre | Mal orden de capas (COPY . . antes de RUN) | Mover
```

```

dependencias antes del código |
| Imagen muy pesada | Usar imagen base :latest completa | Cambiar a :slim o
:alpine |
| Caché no funciona | Cambió un archivo antes de lo esperado | Revisar orden: lo
estable primero |
| docker push rechazado | No hay login o tag mal escrito | docker login y
verificar usuario/imagen:tag |
| Archivos .env en la imagen | Falta .dockerignore | Crear .dockerignore y
reconstruir |
| Error no space left | Imágenes acumuladas en disco | docker system prune -a |

```

👤 **Guión del Docente (Lo que debes decir a los alumnos):**

"Revisemos esta tabla de **Resolución de Errores Frecuentes de la Sesión 2**:

1. **Build lento:** Ocurre si colocas `COPY . .` antes de `RUN pip install`. Solución: Copia primero solo `requirements.txt`.
2. **Imagen pesada:** Usaste la imagen base genérica. Solución: Cambia a `:slim`.
3. **Push rechazado:** Ocurre si no hiciste `docker login` o si el tag no empieza con tu nombre de usuario exacto de Docker Hub.
4. **Archivos sensibles dentro de la imagen:** Te olvidaste de agregar `.env` en tu `.dockerignore`.
5. **Error sin espacio en disco:** Acumulaste imágenes viejas. Solución: Ejecuta `docker system prune -a` para limpiar."

👤 **Acción en Consola / Pizarra:**

- Dejar la tabla de errores visible para la fase de preguntas.

💡 **Tip de Gestión del Aula:**

- Enseñar a diagnosticar leyendo los mensajes de error de la terminal antes de entrar en pánico.

📄 Diapositiva 34: Checklist de Aprendizaje — Sesión 2

Contenido de la PPT:

CHECKLIST DE APRENDIZAJE – SESIÓN 2

- ✓ Puedo escribir un Dockerfile con FROM, RUN, COPY, WORKDIR, ENV, EXPOSE y CMD.
- ✓ Puedo explicar la diferencia entre COPY y ADD, CMD y ENTRYPOINT.
- ✓ Puedo explicar qué es una capa y cómo afecta el orden de las instrucciones.
- ✓ Puedo optimizar un Dockerfile para aprovechar la caché.
- ✓ Puedo usar .dockerignore para excluir archivos del contexto de build.
- ✓ Puedo elegir entre imagen base completa, slim y alpine.
- ✓ Puedo publicar una imagen en Docker Hub con tagging y versionado.
- ✓ Puedo descargar y ejecutar una imagen desde Docker Hub.

👤 **Guión del Docente (Lo que debes decir a los alumnos):**

"Hemos concluido exitosamente la **Sesión 2**. Repasemos nuestro **Checklist de Aprendizaje**: Hoy han aprendido a redactar Dockerfiles profesionales, diferencian COPY de ADD y CMD de ENTRYPOINT, entienden la inmutabilidad de capas y la magia de la caché de build, saben crear un `.dockerignore`, comparan `slim` vs `alpine`, y han publicado y descargado sus propias imágenes en Docker Hub.

¡Excelente trabajo! En la **Sesión 3** aprenderemos a unificar aplicaciones multi-contenedor (Flask + PostgreSQL) utilizando **Docker Compose**. ¡Nos vemos en la siguiente clase!"

Acción en Consola / Pizarra:

- Despedir la clase y recordar resolver el cuestionario de evaluación de la Sesión 2.

Tip de Gestión del Aula:

- Recordar a los alumnos completar la evaluación de 12 preguntas correspondiente a la Sesión 2.